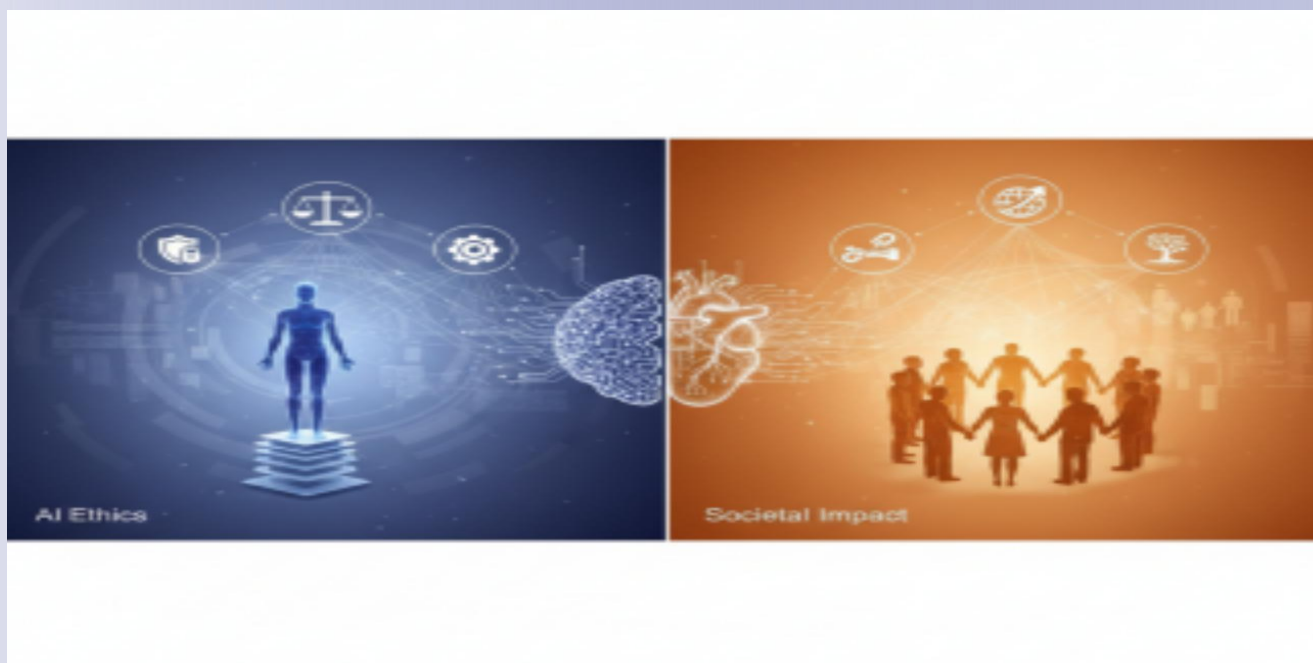
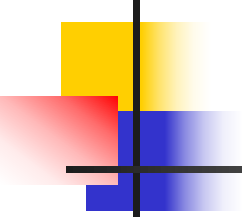


第3章 神经网络基础



湖北大学 计算机学院
王雷春



目 录

3.1 从生物神经元到人工神经元

3.2 单层感知机和多层感知机

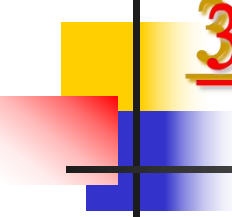
3.3 模型评估和优化

3.4 拟合问题和正则化

3.5 深度学习

3.6 PyTorch简介





3.5 深度学习

3.5.1 概述

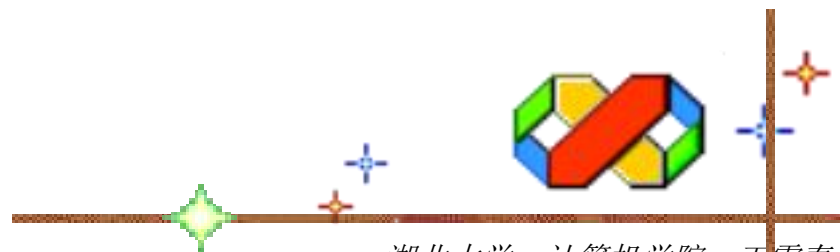
3.5.2 工作原理

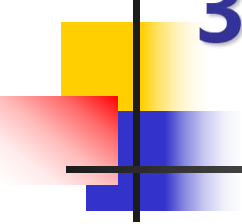
3.5.3 模型结构

3.5.4 深度学习类型

3.5.5 深度学习框架

3.5.6 表达能力与通用性

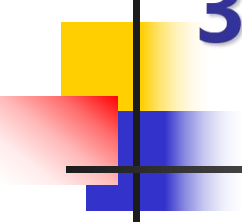




3.5.1 概述

1. 背景与动机

- ◆ (1)传统机器学习依赖人工特征设计和浅层模型，难以自动从高维数据（如图像、语音）中提取复杂抽象特征，性能遇到瓶颈。
- ◆ (2)早期神经网络因梯度消失/爆炸、数据匮乏和算力不足，深层结构难以训练。
- ◆ (3)大数据的涌现、GPU并行计算的普及以及ReLU、残差连接、批归一化等算法突破，共同解决了深度网络的训练难题，使其得以诞生并爆发。



3.5.1 概述

2. 含义

- ◆ 深度学习 (Deep Learning, DL) 是机器学习的一个分支, 通过多层神经网络模拟人脑的层次化信息处理机制, 自动从大规模数据中学习复杂的特征和模式, 从而完成分类、预测、生成等任务。
- ◆ 深度学习的核心思想是通过“深度”网络结构 (多个隐藏层, 通常超过2层) 实现高阶抽象表示, 通过层层递进的方式逐步提取更高层次的抽象特征。例如: 图像识别中, 浅层可能识别边缘、颜色, 深层则识别物体部件或整体。



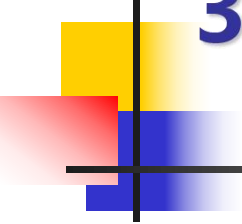
3.5.1 概述

2. 含义

形象化比喻：从原料到建房过程

深度学习过程就像一条从荒山原石开始，自动学习如何破碎、筛分、烧砖、砌墙、搭建楼房及验收的整个过程。

- ◆ (1)岩石破碎（输入层，数据准备与预处理）：将山体中的岩石破碎成较小的石子，去掉不合要求的部分及其中的泥土、树叶。
- ◆ (2)石子筛分（浅层，提取浅层简单特征）：将石子进行筛分，只留下统一合规的小石沙。
- ◆ (3)沙子烧砖（中层，组成中间特征）：将沙子烧成不同规格的红砖。
- ◆ (4)砖块砌墙（深层，形成高级特征）：工人师傅将红砖砌成不同的墙体，组合成不同的房间。

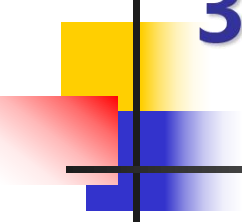


3.5.1 概述

2. 含义

形象化比喻：从原料到建房过程

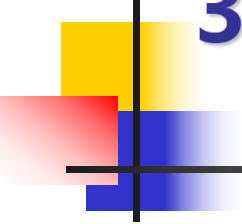
- ◆ (5)搭建楼层（输出层，不同类型的房屋）：在墙体上架设楼板、梁柱，把各个房间连接起来，形成完整的毛坯房。
- ◆ (6)验收整改（评估优化）：对建成的房子进行验收，根据发现的问题进行反向检查问题并整改。



3.5.1 概述

3. 深度学习和传统机器学习

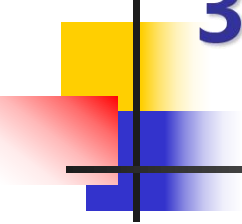
- ◆ (1)特征工程：传统机器学习依赖专家手工设计特征，深度学习自动从数据中学习特征。
- ◆ (2)模型深度：传统机器学习一般是浅层的（如SVM、决策树），深度学习则通常是多层非线性变换（可上百层）。
- ◆ (3)数据要求：传统机器学习只需要较小数据集即可工作，深度学习则需要海量数据才能发挥优势。
- ◆ (4)计算依赖：传统机器学习使用CPU即可，深度学习则需要GPU/TPU才能更好地工作。



3.5.1 概述

4. 主要特点

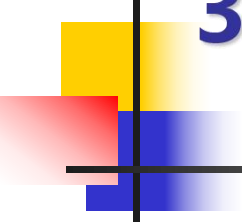
- ◆ (1)自动特征提取。无需人工设计特征，模型直接从原始数据（图像、文本、音频等）中端到端学习有效表示，大幅降低特征工程成本。
- ◆ (2)强大的非线性建模能力。通过多层非线性变换和激活函数，能够逼近任意复杂函数，适用于图像识别、自然语言处理、语音识别等高维复杂问题。
- ◆ (3)层次化抽象表示。通过逐层自动构建从具体到抽象的表示，更好地理解数据的内在结构。
- ◆ (4)数据驱动与可扩展性。性能随数据量和模型规模（层数、参数量）的提升而持续增强，在海量数据场景下显著优于传统机器学习方法。



3.5.2 工作原理

1. 多层非线性变换

- ◆ 深度学习模型通常由输入层、多个隐藏层和输出层构成。每一层由大量神经元组成，每个神经元对输入进行线性加权求和，再经过一个非线性激活函数（如 ReLU、Sigmoid、Tanh）输出结果。
 - ◆ (1)非线性：如果没有激活函数，多层线性变换最终等价于单层线性模型，无法解决非线性问题。激活函数引入非线性，使网络能够拟合任意复杂函数（万能逼近定理）。
 - ◆ (2)多层堆叠：每一层将上一层的输出作为输入，形成层级结构。层数越多，模型可以表达的函数的“复杂度”越高。



3.5.2 工作原理

2. 层次化特征学习

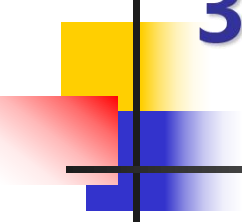
- ◆ 深度学习的核心优势在于自动学习特征的层次结构：
 - ◆ (1)低层：学习局部、简单的模式，如边缘、颜色、纹理。
 - ◆ (2)中层：组合低层特征，形成部件或形状，如眼睛、轮子、音素。
 - ◆ (3)高层：抽象出完整的语义概念，如人脸、汽车、单词。
- ◆ 这种从具体到抽象的逐层转换完全由数据驱动，无需人工设计特征（即“端到端学习”）。



3.5.2 工作原理

3. 训练机制

- ◆ 深度学习模型通过“训练”来确定数亿甚至数万亿个参数（权重和偏置）。训练过程如下：
 - ◆ (1)前向传播：输入数据经过各层计算，得到预测输出。
 - ◆ (2)计算损失：用损失函数（如交叉熵、均方误差）衡量预测值与真实标签之间的差距。
 - ◆ (3)反向传播：利用链式法则，从输出层向输入层逐层计算损失函数对每个参数的梯度。
 - ◆ (4)梯度下降：按照梯度的反方向更新参数（常用优化器如 SGD、Adam），使损失逐步减小。
- ◆ 通过在海量数据上反复迭代这一过程，网络参数逐渐收敛到最优值，从而实现准确的预测。



3.5.2 工作原理

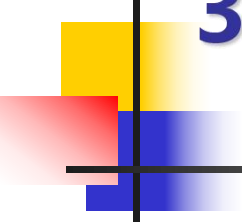
4. 其它关键原理和技术

- ◆ (1)残差连接 (ResNet) : 让输入跳过一层或多层直接加到输出上, 保证梯度能直接流到底层, 缓解退化。
- ◆ (2)批归一化 (Batch Normalization) : 对每一层的输入做标准化, 使梯度分布更稳定, 加速收敛。
- ◆ (3)正则化 (Dropout、权重衰减) : 防止过拟合, 提升泛化能力。

3.5.3 模型结构

1. 基本组成单元

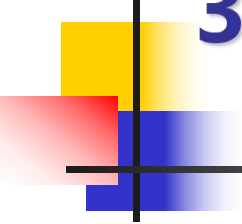
- ◆ 一个深度网络由层（Layer）堆叠而成，每层包含若干神经元（或特征通道）。主要分为：
 - ◆ (1)输入层：接收原始数据（如图像的像素矩阵、文本的词向量序列）。
 - ◆ (2)隐藏层：多个层，进行逐级特征变换。深度就体现在隐藏层的数量上（通常 ≥ 2 ）。
 - ◆ (3)输出层：产生最终预测结果（如分类概率、回归值）。
- ◆ 每层内部核心操作：
输出 = 激活函数(权重 · 输入 + 偏置)



3.5.3 模型结构

2. 常见层类型

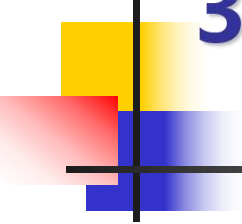
- ◆ (1)全连接层 (FC, Dense) : 每个输入节点与所有输出节点连接, 学习全局组合, 常用于网络末端的分类/回归。
- ◆ (2)卷积层 (Conv2D, Conv1D) : 局部连接+权值共享, 提取空间局部特征, 常用于CNN的前中部。
- ◆ (3)池化层 (Pooling) : 下采样, 降低空间尺寸, 增强平移不变性, 常用于卷积层之后。
- ◆ (4)循环层 (RNN, LSTM, GRU) : 处理序列, 维护隐藏状态 (时间维度的循环连接), 常用于处理文本、时间序列。
- ◆ (5)自注意力层 (Self-Attention) : 计算序列元素之间的关联权重, 并行捕捉长距离依赖, 常用于Transformer的核心。



3.5.3 模型结构

2. 常见层类型

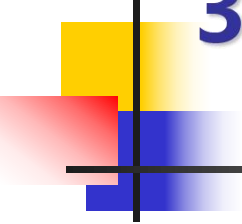
- ◆ (6)归一化层：稳定训练，加速收敛，常用于激活前或激活后。
- ◆ (7)激活层：引入非线性（通常可单独看作层），常用于每个线性变换之后。
- ◆ (8)Dropout层：随机丢弃部分神经元，防止过拟合，常用于全连接层或卷积层之后。



3.5.4 深度学习类型

1. 按网络架构划分

- ◆ (1)前馈神经网络 (FNN): 层间全连接, 信息单向传播, 无反馈, 结构最简单, 如多层感知机。主要用于表格数据、简单分类/回归等任务。
- ◆ (2)卷积神经网络 (CNN): 利用卷积核提取局部特征, 参数共享, 具有平移不变性, 如LeNet、AlexNet、ResNet、EfficientNet和ViT等。主要用于图像分类、目标检测、医学影像等任务。
- ◆ (3)循环神经网络 (RNN) 及其变体: 具有循环连接, 能处理序列数据, 但有长期依赖和梯度消失问题, 如传统RNN、LSTM和GRU等。主要用于文本生成、机器翻译、时间序列预测、语音识别等任务。
- ◆ (4)Transformer: 完全基于自注意力机制, 并行处理序列, 捕获长距离依赖, 如BERT、GPT等。用于NLP任务 (文本分类、问答、翻译) 以及图像、视频、音频等任务。



3.5.4 深度学习类型

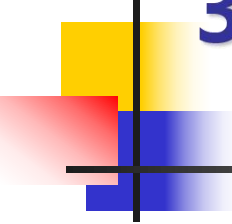
1. 按网络架构划分

- ◆ (5)生成对抗网络(GAN): 生成器与判别器相互博弈, 生成逼真的新数据, 如DCGAN、StyleGAN和CycleGAN等。主要用于图像生成、风格迁移、数据增强、超分辨率等任务。
- ◆ (6)自编码器 (Autoencoder): 编码器压缩输入到低维隐变量, 解码器重构, 如欠完备自编码器、变分自编码器(VAE)、去噪自编码器等。主要用于降维、异常检测、图像去噪、生成模型等任务。
- ◆ (7)图神经网络 (GNN): 处理图结构数据 (节点+边), 通过消息传递聚合邻居信息, 如GCN、GraphSAGE、GAT和GIN等。主要用于社交网络分析、分子性质预测、知识图谱推理、推荐系统等任务。

3.5.4 深度学习类型

2. 按学习范式划分

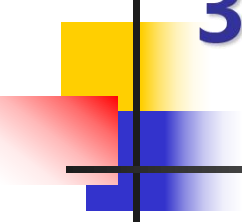
- ◆ (1) 监督学习：使用带标签的 (输入, 输出) 对训练，学习从 $x \rightarrow y$ 的映射，如 CNN 分类、LSTM 序列标注、BERT 微调等。主要用于图像分类、情感分析、语音识别等任务。
- ◆ (2) 无监督学习：只有输入 x ，没有标签 y ，学习数据内在结构或分布，如自编码器、聚类、生成模型 (GAN, VAE) 等。主要用于异常检测、特征提取、数据生成等任务。
- ◆ (3) 半监督学习：少量有标签数据 + 大量无标签数据，结合两者提升性能，如伪标签、一致性正则化 (如 MixMatch) 等。主要用于医疗影像分类、文本分类等任务。
- ◆ (4) 自监督学习：从数据自身构造伪标签，学习通用特征，属于无监督的一种特殊形式，如对比学习 (如 SimCLR)、掩码预测 (如 BERT)。主要用于预训练大模型 (如 GPT)、表征学习等任务。



3.5.4 深度学习类型

2. 按学习范式划分

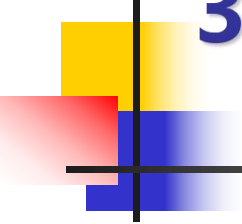
- ◆ (5)强化学习：智能体与环境交互，通过试错和奖励信号学习最优策略，如DQN、PPO和A3C等。主要用于游戏AI（AlphaGo）、机器人控制、自动驾驶决策等任务。
- ◆ (6)迁移学习：将一个任务学到的知识迁移到相关但不同的任务，如预训练+微调（ImageNet→医学图像）。主要用于NLP（BERT到特定领域）、少样本学习等任务。
- ◆ (7)多任务学习：同时学习多个相关任务，共享底层表示，如硬共享、软共享等。主要用于自动驾驶（同时检测车道、行人、标志）等任务。
- ◆ (8)元学习：学习“如何学习”，让模型快速适应新任务（学会学习），如MAML、Reptile和原型网络等。主要用于少样本分类、快速机器人适应等任务。



3.5.5 深度学习框架

1. 含义

- ◆ 深度学习框架（Deep Learning Frameworks）是一种面向神经网络建模的软件库或编程工具。
- ◆ 通过提供高层API和底层自动优化，帮助开发者定义模型结构、自动计算梯度、利用GPU加速，从而高效地完成深度神经网络的训练和推理，让开发者能专注于模型设计本身。
- ◆ 深度学习是目前人工智能的重要发展方向，在开始深度学习项目之前，选择一个合适的框架是非常重要的，选择一个合适的框架能起到事半功倍的效果。

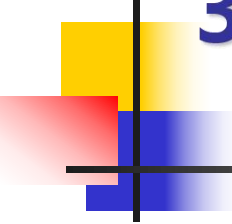


3.5.5 深度学习框架

1. 含义

◆ 深度学习框架的主要作用概括如下：

- ◆ (1)降低编程门槛：自动实现反向传播、矩阵求导和GPU内核调用等功能，节省大量重复代码和调试时间。
- ◆ (2)加速实验迭代：模块化设计和自动微分让你能快速尝试不同网络结构、损失函数和优化器。
- ◆ (3)提升计算效率：自动利用GPU并行能力，同时支持分布式训练（多卡、多机），处理大规模数据。
- ◆ (4)打通全流程：从数据预处理、模型设计、训练、调参到最终部署，框架提供统一工具链。



3.5.5 深度学习框架

2. 常见深度学习框架

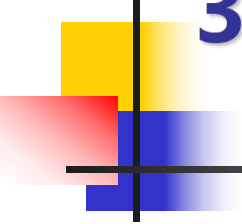
- ◆ (1)Theano。Theano 是第一个有较大影响力的深度学习框架，在2008年诞生于蒙特利尔大学理工学院USA 实验室，为后续的深度学习框架开发奠定了基本的设计方向。Theano 诞生于研究机构，在工程设计上有较大的缺陷。目前，Theano已经停止开发，不推荐将其作为独立的研究工具继续学习。
- ◆ (2)Cafe。Caffe 是一个专注于卷积神经网络（CNN）的深度学习框架，2013年诞生于加州大学伯克利分校。Caffe使用C++语言编写，针对CPU和GPU进行了优化，使得它能够高效地处理大规模的神经网络。Caffe功能相对简单，主要针对计算机视觉领域，其他领域的深度学习任务需要使用其他框架。



3.5.5 深度学习框架

2. 常见深度学习框架

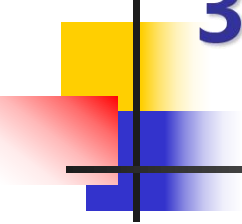
- ◆ (3)Keras。Keras 是一个高级神经网络 API，诞生于谷歌，于2015年6月13日开源，通常作为 TensorFlow 的前端使用。Keras 以用户友好性和模块化设计而闻名，适合快速构建和测试模型。Keras为支持不同的后端进行了过度封装，这使得程序运行较为缓慢，难以新增操作或者获取底层的数据信息。
- ◆ (4)MXNet。MXNet 是 Apache 基金会支持的一个高效、灵活的深度学习框架，于2015 年开源。MXNet 支持多种编程语言接口（如 Python、R 和 Julia），擅长处理大规模数据集和分布式训练任务。MXNet的缺点也很明显，教程不够完善，使用的人不多，导致社区不大。



3.5.5 深度学习框架

2. 常见深度学习框架

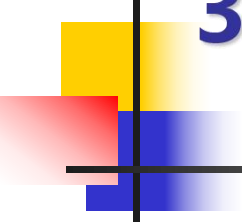
- ◆ (5)TensorFlow。TensorFlow 是由 Google 的GoogleBrain团队开发的深度学习框架，2015年11月10日开源。TensorFlow 使用C++语言开发，支持静态计算图和动态计算图。它具有强大的生态系统和丰富的工具集，适用于生产环境中的模型部署。
- ◆ (6)CNTK。CNTK是微软公司开发的一个深度学习库，2016年1月25日开源。CNTK主要用于高效的训练和推理深度神经网络。CNTK支持多GPU、分布式训练，可以方便地和其他各种框架组合使用。



3.5.5 深度学习框架

2. 常见深度学习框架

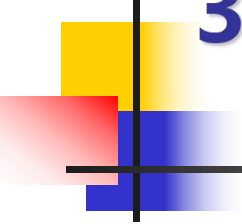
- ◆ (7)PaddlePaddle。PaddlePaddle 是百度开源的深度学习框架，于2016年9月27日开源。支持动静结合的编程范式，同时提供高效的分布式训练能力。该框架在中文社区中有较高的知名度，并且提供了丰富的预训练模型和教程。
- ◆ (8)PyTorch。PyTorch 是 Facebook 开发的深度学习框架，于2017年1月18日开源。PyTorch 的前身是2002年诞生于纽约大学的科学计算库Torch，但对Tensor之上的所有模块进行重构，新增了自动求导系统，不仅更加灵活，支持动态图，而且提供了Python接口，成为最流行的动态图框架。PyTorch 的代码简洁易读，适合学术研究和快速原型开发。



3.5.5 深度学习框架

3. 深度学习框架选择

- ◆ (1) 研究人员、初学者、需要快速迭代的开发者：优先选择PyTorch（代码简洁，社区活跃）。
- ◆ (2) 工业部署/生产环境：优先选择TensorFlow（稳定性高，工具链完整）。
- ◆ (3) 深度学习新手、快速原型开发：优先选择Keras（接口简单）。
- ◆ (4) 国内企业、政府项目，中文NLP开发者：优先选择PaddlePaddle（符合国内数据合规要求）。
- ◆ (5) 高性能计算：优先选择JAX（适合数学密集型任务）。

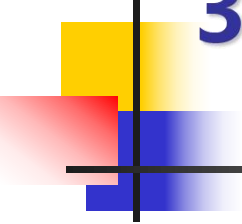


3.5.6 表达能力与通用性

1. 万能近似定理

(1) 定理内容

- ◆ 一个只含一个隐藏层的前馈神经网络，只要隐藏层神经元数量足够多，并选用非常数、有界、单调递增的激活函数（如 Sigmoid 等），就能以任意精度逼近任意定义在实数空间紧致子集上的连续函数。
- ◆ 简单说：单隐藏层的 MLP 理论上可以模拟任何复杂的函数映射，只要给够神经元。

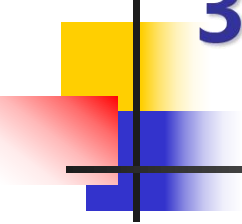


3.5.6 表达能力与通用性

1. 万能近似定理

(2) 定理意义

- ◆ 证明了神经网络的巨大能力：只要参数足够多，神经网络不会因为结构限制而无法拟合复杂映射。
- ◆ 消除早期质疑：在神经网络低潮期，该定理证明了浅层网络的理论能力，促进了研究热情。
- ◆ 为深度学习提供起点：虽然定理只保证“存在性”，但后续工作表明深度网络可以用更少神经元达到相同逼近精度

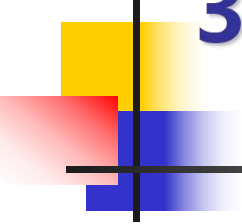


3.5.6 表达能力与通用性

1. 万能近似定理

(3) 定理局限性

- ◆ 只保证存在，不保证可学习。实际训练受限于优化算法、数据和初始化。
- ◆ 需要极大数量的神经元。理论上隐藏层神经元可能多到天文数字，完全不现实。
- ◆ 只针对连续函数。很多实际任务（如图像分类）的输入输出映射不是严格连续的，但可被连续函数近似。
- ◆ 忽略泛化。泛化能力需要另外考虑（正则化、偏差-方差权衡）。

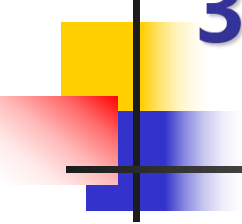


3.5.6 表达能力与通用性

2. 深度与宽度

(1)深度

- ◆ 深度：网络中隐藏层的数量。
- ◆ 深层网络包含多个连续的线性变换+激活函数。
- ◆ 优点：
 - ◆ 表达能力强：能逼近任意复杂函数，尤其擅长高维非线性问题；
 - ◆ 参数效率高：表达同一函数时，深层网络比浅层宽网络需要的参数量指数级减少；
 - ◆ 层次化学习：自动从底层简单特征（边缘、颜色）逐层抽象到高层语义（物体类别）。



3.5.6 表达能力与通用性

2. 深度与宽度

(1)深度

◆ 缺点:

- ◆ 训练困难：易出现梯度消失/爆炸、网络退化（层数越深误差越高）；
- ◆ 资源消耗大：需要更多GPU/TPU内存、训练时间和大规模数据；
- ◆ 技巧要求高：必须借助残差连接、批归一化等特殊方法才能稳定训练；
- ◆ 过拟合与黑箱：数据不足时易过拟合，且模型可解释性差。

3.5.6 表达能力与通用性

2. 深度与宽度

(2) 宽度

- ◆ 宽度：某一层内神经元的数量。
- ◆ 宽层在同一层次上拥有更多并行计算单元。
- ◆ 优点：
 - ◆ 训练相对容易：梯度传播路径短，不易出现梯度消失/爆炸问题，优化更稳定。
 - ◆ 并行计算高效：层内神经元运算可高度并行（矩阵乘法），适合GPU加速。
 - ◆ 捕捉同层多样性：宽层能并行提取同一抽象层次上的多种特征（如不同纹理、形状）。
 - ◆ 缓解深层退化：适度的宽度可提升模型容量，而不必一味增加深度。

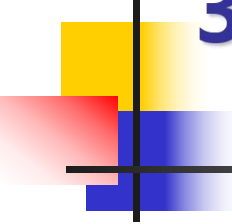
3.5.6 表达能力与通用性

2. 深度与宽度

(2)宽度

◆ 缺点:

- ◆ 参数效率低：表达复杂函数时，宽网络比深网络需要指数级更多的参数量，易浪费计算资源。
- ◆ 过拟合风险高：参数过多，若数据量不足，模型容易记忆噪声，泛化能力差。
- ◆ 特征抽象能力弱：缺乏逐层提炼的层次化结构，难以学习从具体到高级语义的递进表示。
- ◆ 计算开销大：宽度增加导致参数量和FLOPs平方级增长（如全连接层），内存和训练时间显著上升。

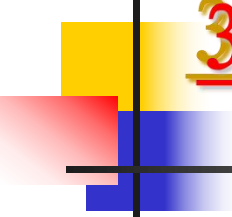


3.5.6 表达能力与通用性

2. 深度与宽度

(3)选择方法

- ◆ 神经网络深度和宽度在选择时通常先深度后宽度，但宽度不能太小。
- ◆ (1)数据量大且复杂（如ImageNet、语音识别等）：优先增加深度，配合残差连接、批归一化等；宽度适中（如每层 64~512）。
- ◆ (2)数据量中等（几千到几万样本）：用中等深度（5~20层），宽度可稍大（如 128~1024），配合强正则化。
- ◆ (3)数据量很小（ < 1000 ）：优先用宽度网络（甚至逻辑回归或浅层MLP），避免深度造成的过拟合和优化困难。
- ◆ (4)需要低延迟推理（实时系统）：减少深度（层数少顺序计算耗时少），适度增加宽度以保持容量，同时利用并行计算。



3.6 PyTorch简介

3.6.1 PyTorch特点

3.6.2 常用模块

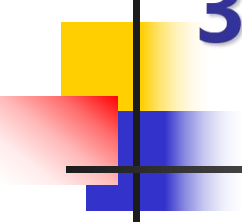
3.6.3 张量基础

3.6.4 张量操作

3.6.5 模型构建

3.6.6 案例

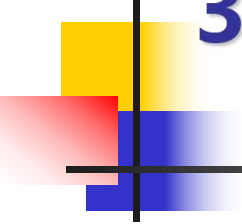




3.6.1 表达能力与通用性

1. PyTorch特点

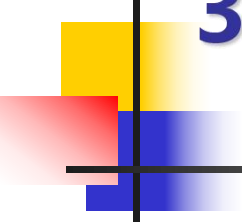
- ◆ (1)动态计算图：计算图随代码执行动态构建，模型结构像普通 Python 程序一样灵活调试与修改。
- ◆ (2)Python优先与NumPy无缝衔接：张量操作接口与 NumPy 几乎一致，深度融入 Python 生态。
- ◆ (3)自动微分（autograd）：自动追踪张量操作并计算梯度，免除手工操作麻烦。
- ◆ (4)模块化网络构建：通过继承 `nn.Module` 并组合预定义层，像搭积木一样清晰搭建神经网络。
- ◆ (5)卓越的调试与开发体验：允许使用标准 Python 调试工具（如 `pdb`）进行调试、检查。



3.6.1 表达能力与通用性

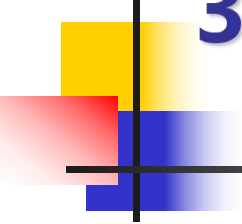
1. PyTorch特点

- ◆ (6)强大的生产部署工具：提供 TorchScript、ONNX 导出和 TorchServe，模型部署应用方便。
- ◆ (7)活跃的生态系统：涌现出 torchvision、transformers、Lightning 等丰富库，满足各领域需求。
- ◆ (8)强大的社区支持和资源。包括：活跃的论坛与讨论组、丰富的教程与文档、开源项目与库等。



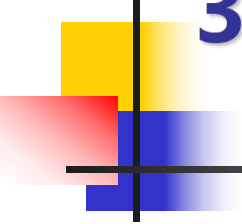
3.6.2 常用模块

- ◆ (1)torch: PyTorch的基石, 提供GPU加速的张量与自动微分, 如张量计算、自动求导等。
- ◆ (2)torch.nn: 所有模型架构的起点, 定义网络“骨架”, 如构建神经网络层、损失函数等。
- ◆ (3)torch.optim: 负责更新模型权重, 让损失函数逐步降低, 提供SGD、Adam等优化算法。
- ◆ (4)torchvision: 视觉任务的“一站式工具箱”, 包括计算机视觉方面的经典数据集 (ImageNet)、模型 (ResNet) 和图像变换等。
- ◆ (5)torchaudio: 处理音频信号的专用工具, 能够进行音频处理, 如加载、变换、常用模型等。



3.6.2 常用模块

- ◆ (6)torchtext: 文本数据的得力助手, 可以进行自然语言处理, 如数据加载、预处理等。已经停止维护了。
- ◆ (7)torchdata: 增强版DataLoader, 构建生产级数据管道, 如高效、可组合的数据加载与处理等。
- ◆ (8)PyTorch Lightning: 将训练逻辑与模型定义分离, 代码更整洁, 如高阶训练框架、自动处理分布式和混合精度等工程细节等。



3.6.3 张量基础

1. 张量简介

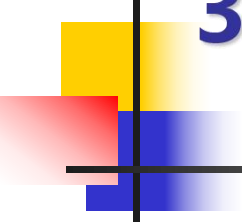
- ◆ 在深度学习中，张量本质上是一个多维数组，是存储和处理数据的基本单位，提供了一种统一、结构化且计算高效的方式来描述和操作复杂数据。
- ◆ 可以把张量看作是NumPy的ndarray的增强版，在PyTorch中张量具有强大的GPU加速计算和自动求导功能。

3.6.3 张量基础

2. 张量类型

表 PyTorch 中的张量（数据）类型

类型	<u>dtype</u>	说明	典型用途
浮点型	torch.float32 或 torch.float	32 位单精度浮点数	最常用
	torch.float64 或 torch.double	64 位双精度浮点数	高精度科学计算
	torch.float16 或 torch.half	16 位半精度浮点数	混合精度训练，节省显存
	torch.bfloat16	脑浮点数	新一代 AI 芯片（如 TPU）支持
整型	torch.uint8	8 位无符号整数	图像数据
	torch.int8	8 位有符号整数	存储小整数
	torch.int16 或 torch.short	16 位整数	存储小整数
	torch.int32 或 torch.int	32 位整数	一般整数计算
	torch.int64 或 torch.long	64 位长整数	索引操作
	torch. <u>bool</u>	布尔类型	掩码、条件判断



3.6.3 张量基础

3. 张量创建

(1)使用`torch.tensor()`类创建张量

- ◆ 用于从已有的 Python 数据（如列表、元组等）创建张量。
- ◆ `torch.tensor(data, dtype=None, device=None, requires_grad=False)`
- ◆ `data`: 初始化数据，可以是一个数、列表、元组、NumPy数组、标量、张量或其它类型。
- ◆ `dtype`: tensor元素的数据类型。
- ◆ `device`: 指定CPU或者是GPU设备，默认是None。
- ◆ `requires_grad`: 是否可以求导，即求梯度，默认是False，即不可导的。

3.6.3 张量基础

3. 张量创建

【例】使用torch.tensor()函数创建张量。

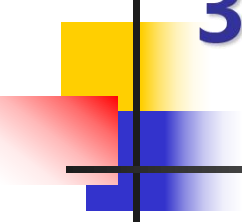
◆ 程序代码及运行结果：

◆ >>> import torch

◆ >>> torch.tensor([]) # 创建一个不包含任何元素的张量
tensor([])

◆ >>> torch.tensor(2) # 通过一个数创建张量
tensor(2)

◆ >>> torch.tensor([1,2]) # 通过一个列表创建张量
tensor([1, 2])

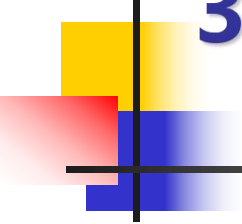


3.6.3 张量基础

3. 张量创建

【例】使用torch.tensor()函数创建张量。

- ◆ `tensor([[1., 2., 3.],
 [4., 5., 6.]], dtype=torch.float64)`
- ◆ `>>> torch.tensor(3.0, requires_grad=True) # 创建可以求梯度张量
tensor(3., requires_grad=True)`



3.6.3 张量基础

3. 张量创建

(2)使用torch中的函数创建张量

- ◆ `torch.arange(start, stop, step)`: 返回初值为start、终值为stop、步长为step的一维张量。
- ◆ `torch.linspace(start, stop, num)`: 返回初值为start、终值为stop、个数为num的等差数列张量。
- ◆ `torch.randperm(n)`: 返回一个范围为[0,n-1]随机排列的张量。

3.6.3 张量基础

3. 张量创建

(2)使用torch中的函数创建张量

- ◆ 【例】使用torch.arange()函数创建张量。
- ◆ 程序代码及运行结果：
- ◆ >>> import torch
- ◆ >>> torch.arange(6) # 不包括终值
tensor([0, 1, 2, 3, 4, 5])
- ◆ >>> torch.arange(2,6,2) # 不包括终值,
torch.LongTensor类型
tensor([2, 4])

3.6.3 张量基础

3. 张量创建

(2)使用torch中的函数创建张量

- ◆ `>>> torch.arange(6,2,-1)` # 包括终值, torch.LongTensor类型
 `tensor([6, 5, 4, 3])`
- ◆ `>>> torch.linspace(1,10,5)` # 元素个数为5, 公差为2.25
 `tensor([ 1.0000, 3.2500, 5.5000, 7.7500, 10.0000])`
- ◆ `>>> torch.randperm(6)` # 随机排列
 `tensor([2, 4, 3, 5, 0, 1])`

3.6.3 张量基础

4. 常用属性

方法或属性	功能	方法或属性	功能
<code>tensor.data</code>	查看张量中的数据	<code>tensor.item()</code>	查看单元素张量的值
<code>tensor.shape</code> 或 <code>tensor.size()</code>	查看张量的形状	<code>tensor.T</code>	返回张量的转置
<code>tensor.dtype</code> 或 <code>tensor.type()</code>	查看张量的数据类型	<code>tensor.requires_grad</code>	是否启用梯度计算
<code>tensor.dim()</code> 或 <code>tensor.ndim</code>	查看张量的维度数	<code>tensor.grad</code>	查看张量的梯度
<code>tensor.numel()</code>	查看张量中的元素个数	<code>tensor.device</code>	查看张量所在的设备

3.6.3 张量基础

4. 常用属性

【例】查看张量属性。

◆ 程序代码及运行结果：

◆ `>>> import torch`

◆ `>>> x = torch.tensor([[1,2,3],[4,5,6]])`

◆ `>>> x.data` # 查看张量x中的数据

```
tensor([[1, 2, 3],  
        [4, 5, 6]])
```

◆ `>>> x.shape` # 查看张量x的形状

```
torch.Size([2, 3])
```

3.6.3 张量基础

4. 常用属性

【例】查看张量属性。

◆ `>>>x.dtype`
`torch.int64`

查看张量x的数据类型

◆ `>>>x.ndim`
`2`

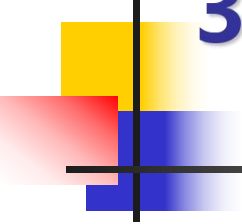
查看张量x的维度数

◆ `>>>x.numel()`
`6`

查看张量x中的元素个数

◆ `>>>x.T`
`tensor([[1, 4],`
 `[2, 5],`
 `[3, 6]])`

返回张量x的转置



3.6.4 张量操作

1. 张量访问

- ◆ 在PyTorch中，张量中的元素访问方式非常灵活，具体格式如下：
- ◆ `tensor`：张量中所有元素。
- ◆ `tensor[切片或整数, 切片或整数]`：张量中dim=0维索引为i所有元素。
(以二维为例)
- ◆ `tensor[:,j]`：张量中dim=1维索引为j所有元素。

3.6.4 张量操作

1. 张量访问和修改

【例】访问张量中的元素。

◆ 程序代码与运行结果：

◆ `>>> import torch`

◆ `>>> x = torch.tensor([[1,2,3],[4,5,6]])` # 创建形状为(2,3)的张量x

◆ `>>> x` # 张量所有元素

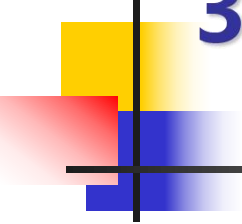
```
tensor([[1, 2, 3],  
        [4, 5, 6]])
```

◆ `>>> x[1]` # dim=0维索引为1的所有元素

```
tensor([4, 5, 6])
```

◆ `>>> x[:,1]` # dim=1维索引为1的所有元素

```
tensor([2, 5])
```



3.6.4 张量操作

2. 张量修改

- ◆ 可以通过张量索引或切片方式对指定张量元素或指定范围的张量元素进行修改，语法格式如下：
- ◆ `tensor[start:stop:step或i, start:stop:step或j]=int或tensor`
- ◆ 参数含义与张量访问中的相同。

3.6.4 张量操作

2. 张量修改

【例】张量修改。

- ◆ 程序代码及运行结果：
- ◆ `>>> import torch`
- ◆ `>>> x = torch.tensor([[1,2,3],[4,5,6]])# 创建形状为(2,3)的张量x`
- ◆ `>>> x[0,0]=11;x[0,1]=12;x[0,2]=13 # 修改张量中单个元素`
- ◆ `>>> x`
`tensor([[11, 12, 13],`
 `[ 4, 5, 6]])`

3.6.4 张量操作

3. 算术操作

【例】张量算术运算。

◆ 程序代码及运行结果：

◆ `import torch`

◆ `x=torch.Tensor([1,2,3])`

创建一个形状为(3,)的张量x

◆ `y=torch.Tensor([4,5,6])`

创建一个形状为(3,)的张量y

◆ `>>>x+y`

张量x和张量y进行加法运算

`tensor([5., 7., 9.])`

◆ `>>>x-y`

张量x和张量y进行减法运算

`tensor([-3., -3., -3.])`

3.6.4 张量操作

4. 矩阵操作

- ◆ `torch.matmul(input, other)` : 矩阵乘法。
- ◆ `tensor.transpose(dim0, dim1)` : 矩阵转置。功能和`tensor.T`相同。

【】 使用`torch.matmul()`函数进行张量运算。

- ◆ 程序代码及运行结果：
- ◆ `>>> import torch`
- ◆ `>>> x=torch.Tensor([[1,2,3],[4,5,6]])` # 创建一个形状为(2,3)的张量
- ◆ `>>> x.transpose(0,1)` # 将张量x转置
`tensor([[1., 4.],`
`[2., 5.],`
`[3., 6.]])`

3.6.4 张量操作

4. 矩阵操作

【】 使用`torch.matmul()`函数进行张量运算。

◆ `>>>y=torch.Tensor([[1,2],[3,4],[5,6]])`

◆ `>>>torch.matmul(x,y)` # 将张量x和张量y进行矩阵乘法
tensor([[22., 28.,
[49., 64.]])

3.6.4 张量操作

5. 形状操作

- ◆ `tensor.size([dim])`: 返回张量指定维度dim的形状。功能和 `tensor.shape` 相同。
- ◆ `tensor.reshape(shape)` 或 `tensor.view(shape)`: 修改张量为指定形状 `shape`。

【例】查看和修改张量形状。

- ◆ 程序代码及运行结果:
- ◆ `>>> import torch`
- ◆ `>>> x=torch.arange(6)` # 创建一个包含6元素的一维张量x
- ◆ `>>> x` # 输出张量x
`tensor([0, 1, 2, 3, 4, 5])`

3.6.4 张量操作

5. 形状操作

【例】查看和修改张量形状。

◆ `>>>x.reshape(2,3)`

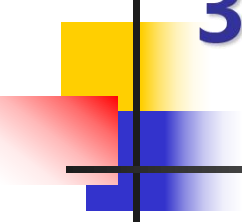
修改张量x的形状为(2,3)

◆ `tensor([[0, 1, 2],
[3, 4, 5]])`

◆ `>>>x.view(2,-1,3)`

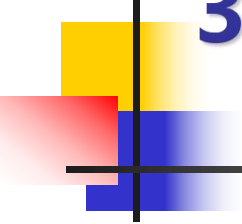
修改张量x的形状为(2,1,3)

◆ `tensor([[[[0, 1, 2]],
[[3, 4, 5]]]])`



3.6.5 模型构建

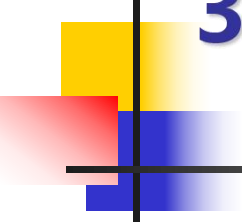
- ◆ PyTorch 提供了几种从简单到复杂、从快速到灵活的模型构建方式，包括：
- ◆ `nn.Module` (子类化)
- ◆ `nn.Sequential` (序列容器)
- ◆ `nn.ModuleList` (模块列表容器) / `nn.ModuleDict` (模块字典容器)。



3.6.5 模型构建

1. 使用 nn.Module 构建模型

- ◆ nn.Module 是所有神经网络模块的基类。
- ◆ 使用 nn.Module 构建模型是 PyTorch 的核心方式：
- ◆ (1) 继承 nn.Module；
- ◆ (2) 在 `__init__`（构造函数）中定义网络层（如 nn.Linear、nn.Conv2d 等）；
- ◆ (3) 在 `forward` 中实现前向传播逻辑。
- ◆ 这种方法高度定制，最为灵活的方式，通过继承 nn.Module 并实现 `forward` 函数，可以自定义复杂的计算逻辑。适用于任何模型，特别是具有跳跃连接、多输入/多输出等复杂结构的网络。



3.6.5 模型构建

1. 使用 nn.Module 构建模型

【例】使用 nn.Module 构建简单全连接网络（MLP）。

- ◆ 分析：
- ◆ (1) 模型包含一个隐藏层（20 个神经元）和 ReLU 激活函数，输出层为 2 个神经元（可用于二分类）。
- ◆ (2) 若用于多分类，可将 output_dim 改为类别数，并结合 nn.CrossEntropyLoss（模型内部无需 Softmax）。
- ◆ (3) 若用于回归，将 output_dim 改为 1，损失函数使用 nn.MSELoss。

3.6.5 模型构建

1. 使用 nn.Module 构建模型

【例】使用 nn.Module 构建简单全连接网络（MLP）。

◆ 程序代码：

◆ import torch

◆ import torch.nn as nn

◆ # 定义MLP模型

◆ class SimpleMLP(nn.Module):

◆ def __init__(self, input_dim=10, hidden_dim=20, output_dim=2):

◆ super(SimpleMLP, self).__init__()

◆ self.fc1 = nn.Linear(input_dim, hidden_dim) # 线性层

◆ self.relu = nn.ReLU() # 激活函数

◆ self.fc2 = nn.Linear(hidden_dim, output_dim) # 线性层

3.6.5 模型构建

1. 使用 nn.Module 构建模型

【例】使用 nn.Module 构建简单全连接网络（MLP）。

- ◆ # 重写 forward(self, x) 方法
- ◆ def forward(self, x):
- ◆ x = self.fc1(x)
- ◆ x = self.relu(x)
- ◆ x = self.fc2(x)
- ◆ return x

- ◆ # 实例化模型
- ◆ model = SimpleMLP(input_dim=10, hidden_dim=20, output_dim=2)

3.6.5 模型构建

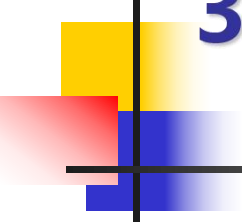
1. 使用 nn.Module 构建模型

【例】使用 nn.Module 构建简单全连接网络（MLP）。

- ◆ # 测试
- ◆ `dummy_input = torch.randn(4, 10)` # 通过随机函数 `torch.randn()` 生成输入数据
- ◆ `output = model(dummy_input)` # 通过模型生成输出数据
- ◆ `print(output.shape)` # 输出结果

运行结果：

- ◆ `torch.Size([4, 2])`



3.6.5 模型构建

2. 使用 `nn.Sequential` 构建模型

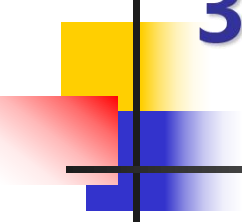
- ◆ 使用`nn.Sequential`是 PyTorch中快速构建顺序式神经网络的最简洁方式。
- ◆ 这种方法无需定义 `forward`方法，因为`nn.Sequential`内部已实现 `forward`，只需按顺序列出层，它会按照你添加层的顺序自动执行前向传播；但是无法处理跳跃连接（如 ResNet），不能共享层或有多输入/输出，调试时无法在层之间插入复杂逻辑。
- ◆ 这种方法适用于结构简单的直连网络，如 快速原型验证、LeNet、简单的MLP等；若需要多输入、多输出或复杂控制流，仍需继承 `nn.Module` 并自定义 `forward`。

3.6.5 模型构建

2. 使用 `nn.Sequential` 构建模型

【例】 用`nn.Sequential`构建一个简单的3层全连接网络。

- ◆ 分析：
- ◆ (1)模型由3个线性层和中间的ReLU激活函数组成；
- ◆ (2)`input_dim`：输入特征维度；
- ◆ (3)`hidden_dim`：隐藏层维度；
- ◆ (4)`output_dim`：输出维度；
- ◆ (5)输入：`batch_size × input_dim` 的张量；
- ◆ (6)输出：`batch_size × output_dim`。



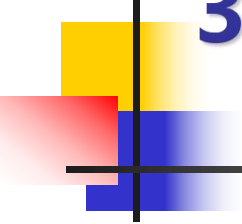
3.6.5 模型构建

2. 使用 nn.Sequential 构建模型

程序代码：

- ◆ `import torch`
- ◆ `import torch.nn as nn`

- ◆ `input_dim = 10` # 输入特征维度
- ◆ `hidden_dim = 20` # 隐藏层维度
- ◆ `output_dim = 1` # 输出维度

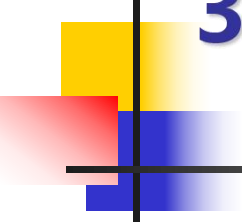


3.6.5 模型构建

2. 使用 nn.Sequential 构建模型

程序代码：

```
◆ # 构建一个3层全连接网络
◆ model = nn.Sequential(
◆     nn.Linear(input_dim, hidden_dim),      # 线性层
◆     nn.ReLU(),                             # 激活函数
◆     nn.Linear(hidden_dim, hidden_dim),      # 线性层
◆     nn.ReLU(),                             # 激活函数
◆     nn.Linear(hidden_dim, output_dim)       # 线性层
◆ )
```



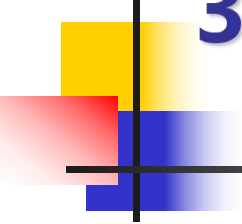
3.6.5 模型构建

2. 使用 nn.Sequential 构建模型

程序代码：

- ◆ # 查看模型结构
- ◆ `print(model)`

- ◆ # 测试前向传播
- ◆ `dummy_input = torch.randn(5, input_dim)`
- ◆ `output = model(dummy_input)`
- ◆ `print(f"输出形状: {output.shape}")`



3.6.5 模型构建

2. 使用 nn.Sequential 构建模型

运行结果:

- ◆ Sequential(
 - ◆ (0): Linear(in_features=10, out_features=20, bias=True)
 - ◆ (1): ReLU()
 - ◆ (2): Linear(in_features=20, out_features=20, bias=True)
 - ◆ (3): ReLU()
 - ◆ (4): Linear(in_features=20, out_features=1, bias=True)
 - ◆)
- ◆ 输出形状: torch.Size([5, 1])

3.6.5 模型构建

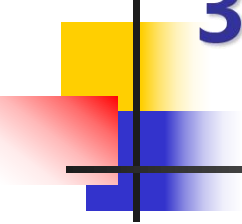
3. 使用nn.ModuleList构建模型

◆ nn.ModuleList

- ◆ 一个可以容纳多个子模块的列表，它会自动注册其中的模块，使得参数可以被优化器发现。
- ◆ 适用需要动态添加多个相同结构层（如残差块、MLP 层）场景。

◆ nn.ModuleDict

- ◆ 一个字典容器，可以用键名来访问子模块。
 - ◆ 适合需要根据不同条件选择不同处理分支的模型（例如根据配置选择不同的特征提取器）。
- ◆ 两者都不会自动定义前向传播，需要你在 forward 中手动编写遍历或索引逻辑。它们主要用于组织和管理子模块，让模型结构更清晰、更易动态修改



3.6.5 模型构建

3. 使用nn.ModuleList构建模型

【例】使用nn.ModuleList动态构建一个可变深度的全连接网络。

- ◆ 分析：
- ◆ (1)通过 self.layers.append() 动态添加层，代码简洁且易于修改网络深度；
- ◆ (2)前向传播时遍历 ModuleList 依次应用每一层；
- ◆ (3)所有 nn.Linear 的参数都会被自动注册。

3.6.5 模型构建

3. 使用nn.ModuleList构建模型

程序代码：

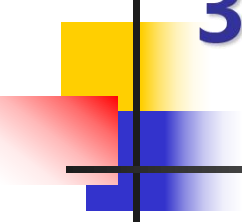
- ◆ import torch
- ◆ import torch.nn as nn
- ◆ # 创建动态隐藏层的MLP
- ◆ class DynamicMLP(nn.Module):
- ◆ def __init__(self, input_dim, hidden_dims, output_dim):
- ◆ super().__init__()
- ◆ self.layers = nn.ModuleList() # 创建 ModuleList

3.6.5 模型构建

3. 使用nn.ModuleList构建模型

程序代码：

- ◆ # 动态添加隐藏层
- ◆ prev_dim = input_dim
- ◆ for h_dim in hidden_dims:
- ◆ self.layers.append(nn.Linear(prev_dim, h_dim))
- ◆ self.layers.append(nn.ReLU())
- ◆ prev_dim = h_dim
- ◆ # 输出层
- ◆ self.layers.append(nn.Linear(prev_dim, output_dim))



3.6.5 模型构建

3. 使用nn.ModuleList构建模型

程序代码：

- ◆ `def forward(self, x):`
- ◆ `for layer in self.layers:`
- ◆ `x = layer(x)`
- ◆ `return x`

3.6.5 模型构建

3. 使用nn.ModuleList构建模型

程序代码：

- ◆ # 使用示例
- ◆ `model = DynamicMLP(input_dim=10, hidden_dims=[64, 32], output_dim=1)`
- ◆ `print(model)`

- ◆ `dummy_input = torch.randn(4, 10)`
- ◆ `output = model(dummy_input)`
- ◆ `print(output.shape) # torch.Size([4, 1])`

3.6.5 模型构建

3. 使用nn.ModuleList构建模型

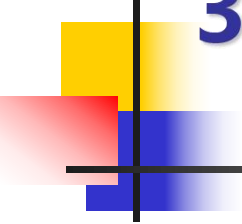
运行结果：

- ◆ DynamicMLP(
 - ◆ (layers): ModuleList(
 - ◆ (0): Linear(in_features=10, out_features=64, bias=True)
 - ◆ (1): ReLU()
 - ◆ (2): Linear(in_features=64, out_features=32, bias=True)
 - ◆ (3): ReLU()
 - ◆ (4): Linear(in_features=32, out_features=1, bias=True)
 - ◆)
- ◆)
- ◆ torch.Size([4, 1])

3.6.6 案例

1. MNIST数据集简介

- ◆ MNIST数据集由Yann LeCun等人在1998年创建，是深度学习领域的“Hello World”，一个被高度标准化和广泛使用的手写数字图像数据库。它由70,000张28x28像素的灰度图像组成，包含从0到9的十个数字类别。
- ◆ MNIST的构成非常简单，其核心构成如下：
 - ◆ (1)训练集：60,000张，用于模型学习。
 - ◆ (2)测试集：10,000张，用于评估模型效果。
 - ◆ (3)图像详情：28x28像素的手写数字灰度图像，像素值范围是0-255。
 - ◆ (4)标签：每张图片都有对应的标签（0-9），标注了图中手写的数字。



3.6.6 案例

2. 手写数字识别

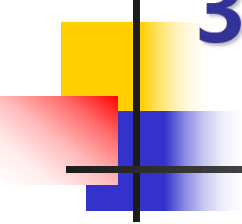
【例】使用MLP实现MNIST的手写数字识别。

分析：

- ◆ 本案例数据集选择MNIST数据集基本版。为了加快程序运行速度，可提前下载MNIST数据集，然后放在指定目录下。

具体步骤：

- ◆ (1)准备数据：首先下载MNIST图像数据到指定路径；然后将输入的PIL 图像或 NumPy 数组转换为 PyTorch 张量，使用 transforms.Compose()函数对张量进行标准化；最后将训练集和测试集分成多个小批量（batch），每次迭代返回一个 batch 的数据。



3.6.6 案例

2. 手写数字识别

【例】使用MLP实现MNIST的手写数字识别。

具体步骤：

- ◆ (2)定义模型：通过继承nn.Module定义一个简单的MLP模型。
- ◆ (3)初始化模型、损失函数、优化器：对模型进行初始化；选择合适的损失函数；确定适当的优化器，并设置学习率。
- ◆ (4)模型训练：将模型在指定的训练集上训练若干epoch。
- ◆ (5)模型预测：在测试集上对指定的数字图片进行预测。
- ◆ (6)输出结果：输出数字图片的真实结果和预测结果。

3.6.6 案例

2. 手写数字识别

【例】使用MLP实现MNIST的手写数字识别。

运行结果：

- ◆ Epoch 1 completed
- ◆ Epoch 2 completed
- ◆ Epoch 3 completed
- ◆ Epoch 4 completed
- ◆ Epoch 5 completed
- ◆ Epoch 6 completed
- ◆ 预测的数字: 7
- ◆ 真实标签: 7